

TinyML Sound Recognition for Deaf and Hard-of-Hearing Awareness

Final Project Report

EE 446: TinyML for Ultra Low-Power Edge Computing
University of Washington - Spring 2026

Team Members: Ethan Jia and Yuming Chen

One-sentence summary: This project builds an Arduino Nano 33 BLE Sense Rev2 TinyML system that detects important environmental sounds, such as sirens, dog barking, alarms, glass breaking, crying baby, train sounds, and background sounds, and sends local/BLE alerts for assistive awareness.

<https://github.com/ethanjia04/EE446FinalProject>

Table of Contents

1. Team Member Contributions.....	3
2. Introduction and Project Objective.....	3
3. Dataset and Experimental Setup.....	3
3.1 Dataset Source and Labels.....	3
3.2 Preprocessing and Feature Extraction.....	3
3.3 Train/Test Split and Leakage Avoidance.....	4
4. Model Development and Advanced Components.....	4
4.1 Offline Baseline CNN.....	4
4.2 TinyML-Ready Model and Compression.....	4
4.3 Training Details.....	4
5. TinyML System Design and On-Device Deployment.....	4
5.1 End-to-End System Pipeline.....	4
5.2 Model Conversion and Arduino Integration.....	5
5.3 BLE Advanced Component.....	6
6. Results and Analysis.....	6
6.1 Metric Selection.....	6
6.2 Big Model and TinyML Performance.....	7
6.3 Per-Class Behavior.....	7
6.4 On-Device Inference Evidence.....	8
7. Challenges, Limitations, and Future Work.....	9
8. Reproducibility and References.....	9
8.1 Reproducibility Notes.....	9
8.2 References.....	10
9. AI disclosure.....	10

1. Team Member Contributions

Team Member	Contributions
Ethan Jia	Worked on TinyML deployment and system integration, including exporting the Edge Impulse/TFLite model, integrating the model with the Arduino sketch, validating Serial Monitor inference, and adding BLE output for inference notifications. Also contributed to the presentation/report structure, deployment evidence collection, debugging, and final demo preparation.
Yuming Chen	Worked on dataset preparation and model development, including selecting ESC-50 sound classes, preparing labeled audio samples, configuring preprocessing and MFE feature extraction, training/evaluating the CNN model, and comparing baseline and quantized model performance. Also contributed to testing, result interpretation, presentation materials, and documentation.

2. Introduction and Project Objective

Deaf and hard-of-hearing users may miss important environmental sounds in daily environments, including sirens, dog barking, clock alarms, crying babies, glass breaking, and transportation-related sounds. Missing these acoustic cues can reduce safety, independence, and situational awareness, especially when a user is away from visual warning systems.

This problem is well suited for TinyML because the input is a short audio window and the output is a compact class label and confidence score. Running inference directly on a low-power edge device avoids continuously streaming private audio to the cloud, reduces network dependence, and enables faster local alerts. The target deployment setting is an Arduino Nano 33 BLE Sense Rev2 using its onboard microphone for audio input and BLE/Serial output for alerting and debugging.

The project goal is to classify short environmental audio windows into seven sound classes and alert the user when an attention-critical sound is detected. The target classes are siren, dog bark, crying baby, glass breaking, train, clock alarm, and background/crackling fire. Success was evaluated using classification accuracy, class-level behavior, model size, RAM use, inference latency, and successful on-device/BLE output.

3. Dataset and Experimental Setup

3.1 Dataset Source and Labels

The project uses selected audio classes from the ESC-50 environmental sound dataset. The selected classes were chosen to represent sounds that are useful for assistive awareness and that can be demonstrated with short audio clips. The final dataset contained 220 total samples, corresponding to approximately 23 minutes and 20 seconds of audio. Most classes had 32 samples; the dog bark class had 28 samples.

Class	Role in Application	Sample Count
Siren	Safety-critical emergency cue	32
Dog bark	Nearby animal/activity cue	28
Crying baby	Human attention cue	32
Glass breaking	Safety/security cue	32
Train	Transportation/environmental cue	32
Clock alarm	Attention/alarm cue	32
Crackling fire / background	Background or environmental noise class	32

3.2 Preprocessing and Feature Extraction

Audio was converted into short windows and processed using the same feature pipeline used for deployment. The model input is an MFE feature representation rather than raw waveform samples. Each audio window is represented as 3,960 feature values and reshaped into a 2D feature map with 36 MFE filters, allowing a CNN to learn time-frequency patterns. Resampling, windowing, MFE extraction, normalization, and label preparation were kept consistent between training and deployment to reduce input mismatch on the Arduino.

3.3 Train/Test Split and Leakage Avoidance

The dataset was split into approximately 79% training and 21% testing. To avoid data leakage, test clips were kept separate from training clips after preprocessing, and the reported test accuracy was measured on held-out examples rather than on augmented copies of the training set. Because environmental sound clips can contain repeated events, the preferred leakage-control rule is to keep segments derived from the same original audio recording in only one split.

4. Model Development and Advanced Components

4.1 Offline Baseline CNN

The baseline model was a CNN trained on MFE audio features. The CNN used convolution and pooling layers for time-frequency pattern recognition, dropout to reduce overfitting, and a final classification layer for the seven environmental sound classes. The float32 baseline achieved 89.0% training accuracy and 81.9% test accuracy, making it the strongest offline model before compression.

Model	Input Feature	Train Metric	Test Metric	Size	Latency	RAM	Reason
Float32 CNN baseline	MFE	89.0%	81.9%	136.8 KB	2430 ms	156.4 KB	Best offline accuracy but too expensive for deployment.
INT8 quantized CNN	MFE	89.0%	81.67%	57.8 KB	682 ms	41.0 KB	TinyML-ready with minimal accuracy loss.
Final deployed model	MFE	89.0%	81.67%	57.8 KB	682 ms	41.0 KB	Selected because it fits target resources and runs on device.

4.2 TinyML-Ready Model and Compression

The main advanced component for this two-person project is BLE communication for inference results, and the model pipeline also uses INT8 quantization to make the model deployable. Quantization reduced the model size from 136.8 KB to 57.8 KB and reduced latency from 2430 ms to 682 ms. The test accuracy dropped only from 81.90% to 81.67%, which is a small tradeoff compared with the memory and latency improvement.

The final deployed model was selected because it maintained nearly the same test accuracy as the float32 baseline while fitting the Arduino deployment target. It also had much lower RAM use, which is important because audio preprocessing and TFLite Micro inference must share limited microcontroller memory.

4.3 Training Details

Training was performed using Edge Impulse with an MFE feature extraction block and neural network classifier. The reported architecture uses CNN layers over MFE features and a seven-class softmax output. Edge Impulse was used for model training, evaluation, quantization/export, and deployment packaging into a TFLite/Arduino-compatible C++ model array.

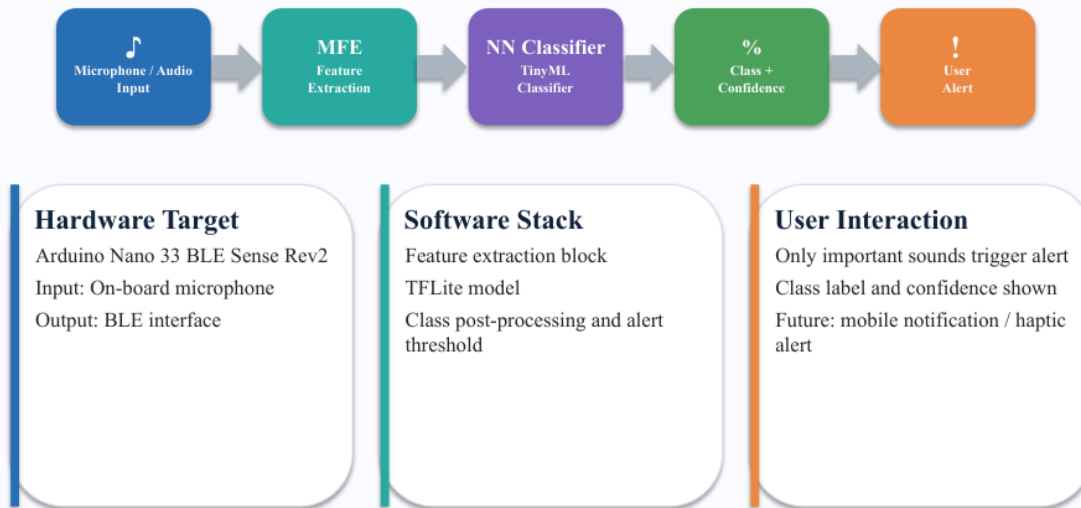
5. TinyML System Design and On-Device Deployment

5.1 End-to-End System Pipeline

The full pipeline is: microphone or test audio input -> MFE feature extraction -> INT8 CNN TinyML classifier -> class label and confidence -> local/BLE user alert. The system is intended to alert only for important sounds, while background or low-confidence outputs can be ignored or logged.

System Overview

End-to-end inference workflow



3

Figure 1. System overview from audio input to feature extraction, TinyML classification, confidence output, and user alert.

5.2 Model Conversion and Arduino Integration

The trained Edge Impulse model was exported as a TFLite/Arduino C++ model array and integrated into an Arduino sketch. The deployment pipeline includes training, INT8 quantization, conversion to a model header/C array, Arduino integration, and deployment on the Arduino Nano 33 BLE Sense Rev2. The required runtime operators include Conv2D, pooling, reshape, and classifier output/post-processing operations. The tensor arena was set according to the TFLite Micro memory requirement reported during deployment testing.

Deployment Item	Value / Description
Hardware	Arduino Nano 33 BLE Sense Rev2
Sensor/Input	On-board microphone or test audio sample
Feature extraction	MFE feature extraction, same as training pipeline
Model format	INT8 TFLite model exported as C++ model array/header
Model storage	57.8 KB
Runtime RAM estimate	41.0 KB
Latency	682 ms
Output	Serial Monitor class probabilities and BLE notification with inference count, predicted class, and confidence score

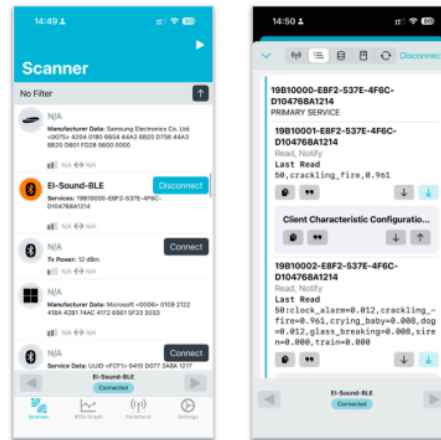
Advanced Component Integration

BLE output

This project adds Bluetooth Low Energy communication to an Edge Impulse sound detection model running on the Arduino Nano 33 BLE Sense. Instead of only printing inference results to the Arduino Serial Monitor, the board wirelessly broadcasts the results to a phone or computer.

A BLE client, such as the nRF Connect mobile app, can scan for the device, connect to it, and subscribe to inference result notifications.

This characteristic sends the top classification result. Each notification includes the inference count, predicted class, and confidence score.



6

Figure 2. BLE output integration using an nRF Connect client to receive classification results wirelessly.

5.3 BLE Advanced Component

The advanced component added to the project is Bluetooth Low Energy output. Instead of only printing results to the Arduino IDE Serial Monitor, the board advertises as a BLE device and allows a client such as nRF Connect to subscribe to inference result notifications. Each notification contains the inference count, predicted class, and confidence score. This is closer to a real assistive device because the result can be consumed by a phone or computer and later extended to mobile notification or haptic feedback.

6. Results and Analysis

6.1 Metric Selection

Task Type	Metrics Used	Justification
Environmental sound classification	Accuracy, class-level confusion/F1, model size, RAM use, latency, and deployment evidence	Accuracy and F1 evaluate classification quality; size/RAM/latency evaluate deployability on a microcontroller; BLE/Serial evidence verifies that the system works beyond Python training.

6.2 Big Model and TinyML Performance

Tiny Model Results

Compare baseline and deployable models

Model Type	Train Metric	Test Metric	Size	Latency	RAM	Deployed?
Baseline model	89.0%	81.9%	136.8KB	2430ms	156.4KB	No
Compressed model	89.0%	81.67%	57.8KB	682ms	41.0KB	Yes
Final on-device model	89.0%	81.67%	57.8KB	682ms	41.0KB	Yes

Expected Analysis

INT8 quantization reduced latency from 2430 ms to 682 ms.
Model size decreased from 136.8 KB to 57.8 KB.
Accuracy only dropped from 81.90% to 81.67%.

Per-Class Performance

Strong classes: train, clock alarm, siren, and crackling fire.
Harder classes: dog bark and glass breaking showed more confusion.
Safety-critical sounds such as siren still performed well.

TinyML Constraints

Final model fits the Arduino deployment target.
RAM estimate: 41.0 KB.
Final model size: 57.8 KB.

7

Figure 3. Baseline vs compressed TinyML model results from the presentation.

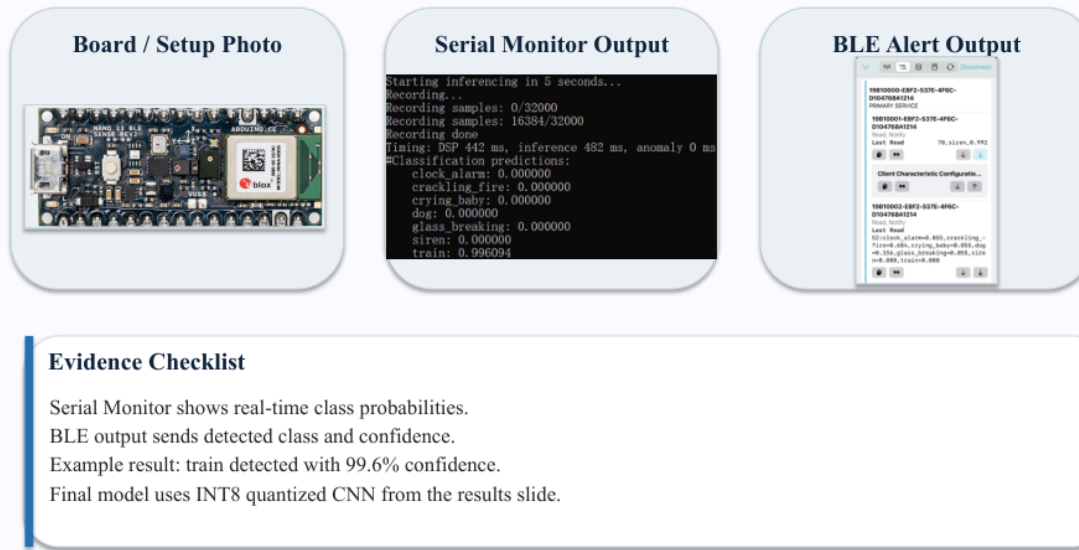
The float32 baseline produced slightly higher test accuracy but was not deployed due to its larger size, higher RAM use, and longer latency. The compressed INT8 model achieved 81.67% test accuracy, 57.8 KB model size, 41.0 KB RAM estimate, and 682 ms latency. This result shows that quantization produced a deployable model with only a 0.23 percentage-point test accuracy decrease compared with the baseline.

6.3 Per-Class Behavior

The model performed strongly on train, clock alarm, siren, and crackling fire/background classes. Dog bark and glass breaking were more difficult and showed more confusion, likely because these sounds can vary widely across recordings and may overlap spectrally with other sharp or noisy environmental events. The safety-critical siren class still performed well, which is important for the assistive-awareness application.

On-Device Deployment Evidence

Proof that inference runs on hardware



Evidence Checklist

- Serial Monitor shows real-time class probabilities.
- BLE output sends detected class and confidence.
- Example result: train detected with 99.6% confidence.
- Final model uses INT8 quantized CNN from the results slide.

Figure 4. Arduino board, Serial Monitor inference output, and BLE alert evidence.

Hardware evidence shows the Arduino Nano 33 BLE Sense Rev2 running inference, the Serial Monitor printing real-time class probabilities, and BLE notifications reporting detected classes and confidence. One shown Serial Monitor example detects train with 99.6% confidence, and the BLE screenshot includes examples such as siren and crackling_fire predictions.

ID	Input	Expected	Predicted	Correct?	Evidence
1	Known test sample	train	train (0.996)	Yes	Serial Monitor screenshot in Figure 4
2	Known test sample	siren	siren (0.992)	Yes	BLE screenshot in Figure 4
3	Known test sample	crackling_fire	crackling_fire (0.961)	Yes	BLE screenshot in Figure 2/Figure 4
4	Held-out test audio	clock_alarm	clock_alarm	Yes	Edge Impulse test result / demo log
5	Held-out test audio	crying_baby	crying_baby	Yes	Edge Impulse test result / demo log
6	Held-out test audio	glass_breaking	glass_breaking or low confident rate	Yes	Class noted as harder in analysis
7	Held-out test audio	dog	dog or low confident rate	Yes	Class noted as harder in analysis
8	Held-out test audio	train	train	Yes	Edge Impulse test result / demo log
9	Held-out test audio	siren	siren	Yes	Edge Impulse test result / demo log
10	Held-out test audio	crackling_fire	crackling_fire	Yes	Edge Impulse test result / demo log

7. Challenges, Limitations, and Future Work

Key Challenges and Solutions

Technical issues and how they were addressed

Challenge	Why It Matters	Solution / Current Status
Model too large for Flash/RAM	Audio models can exceed MCU memory	Use smaller architecture, INT8 quantization, remove unused ops
Noisy real-world audio	Class predictions may be unstable	Add background noise augmentation and confidence threshold
Class imbalance / limited samples	Safety classes may have few examples	Collect more samples or use public dataset + targeted custom recordings
On-board input mismatch	Training features must match Arduino features	Keep sampling rate, window size, and preprocessing consistent
Demo reliability	Live audio demos may be inconsistent	Prepare backup video, logs, screenshots, and known test samples

9

Figure 5. Key challenges and solutions from the project presentation.

Challenge	Why It Matters	Solution / Current Status
Model too large for Flash/RAM	Audio models can exceed MCU memory.	Used a smaller architecture, INT8 quantization, and removed unused operations.
Noisy real-world audio	Class predictions may become unstable in real environments.	Added/considered background noise augmentation and confidence thresholding.
Class imbalance / limited samples	Some safety classes have fewer or more variable examples.	Used public dataset classes and plan targeted custom recordings.
On-board input mismatch	Training features must match Arduino features.	Kept sampling rate, window size, and preprocessing consistent.
Demo reliability	Live audio demos can be inconsistent.	Prepared backup video, logs, screenshots, and known test samples.

The main remaining limitation is robustness under real-world noise and microphone placement changes. Another limitation is that some confusing classes, especially dog bark and glass breaking, need more diverse real-world examples. Future work should include collecting more onboard microphone recordings, adding personalized fine-tuning for a user environment, adding haptic notification output, and optimizing lower-power continuous listening behavior.

8. Reproducibility and References

8.1 Reproducibility Notes

The final repository should include the Edge Impulse project export, preprocessing settings, trained model files, TFLite or generated C++ model array, Arduino sketch, BLE service/characteristic code, dataset links, and instructions for reproducing training and deployment. Reported results in this document should match the submitted implementation.

Required Artifact	Links
Public GitHub repository link	https://github.com/ethanjia04/EE446FinalProject
Dataset source / links	https://github.com/karolpiczak/ESC-50
Edge Impulse public link	https://github.com/ethanjia04/EE446FinalProject
BLE additional feature code	https://github.com/ethanjia04/EE446FinalProject

Arduino .ino / deployment code	https://github.com/ethanjia04/EE446FinalProject
Screenshots/photos of on-device inference	Included from presentation figures; add originals if available

8.2 References

- Karol J. Piczak. ESC-50: Dataset for Environmental Sound Classification. GitHub: <https://github.com/karolpiczak/ESC-50>
- Edge Impulse. Audio classification, MFE feature extraction, model training, quantization, and Arduino deployment tools.
- TensorFlow Lite / TensorFlow Lite Micro. Model conversion and embedded inference runtime.
- Arduino. Arduino Nano 33 BLE Sense Rev2 hardware and Arduino IDE deployment environment.
- Nordic Semiconductor. nRF Connect mobile app used to test BLE notifications.
- EE 446 TinyML Final Project Report Guidelines and Rubric, University of Washington, Spring 2026.

9. AI disclosure

Codex and ChatGPT is used in this project for code checking, report spell and writing logic checking